AI ASSIGNMENT-9.1

NAME:N.Vaishnavi

BATCH:44

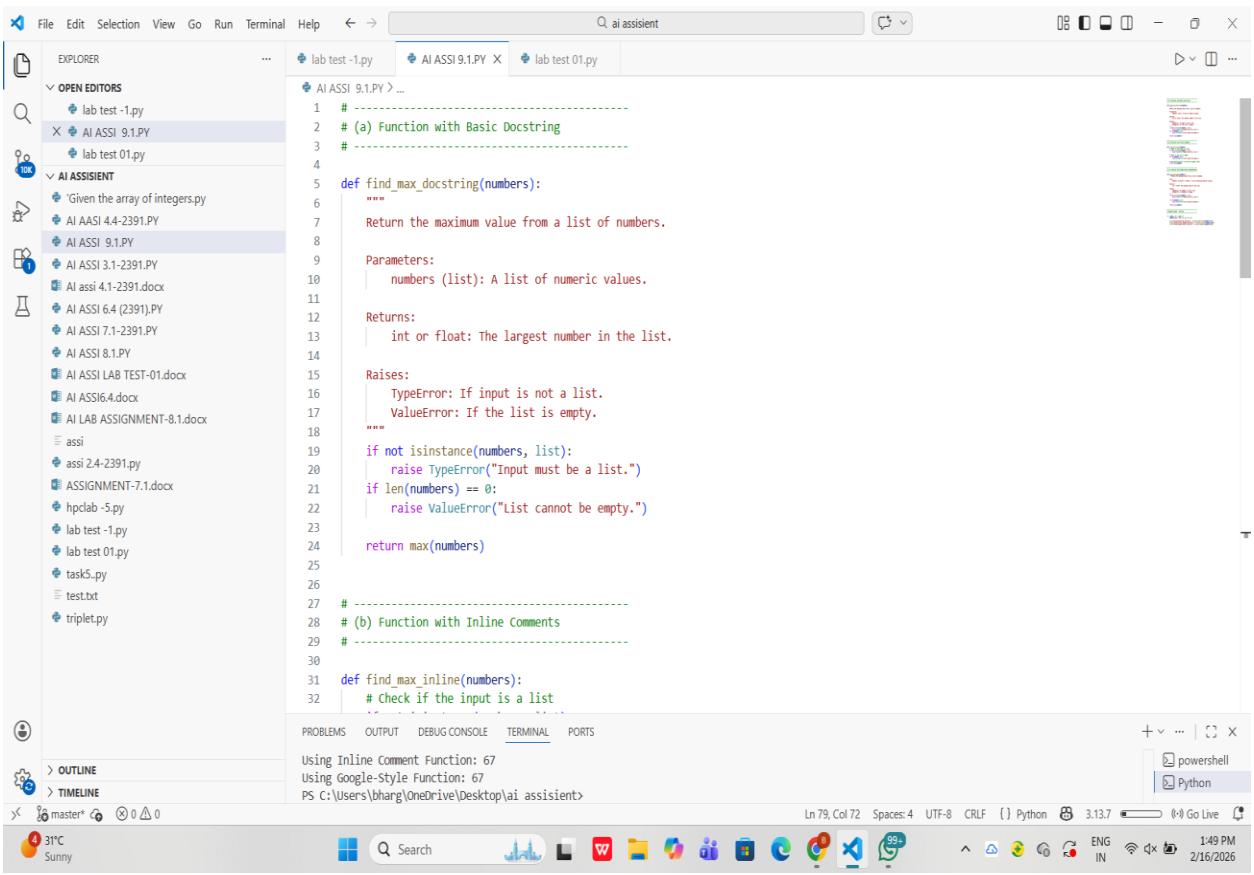
Hall.no:2303A52405

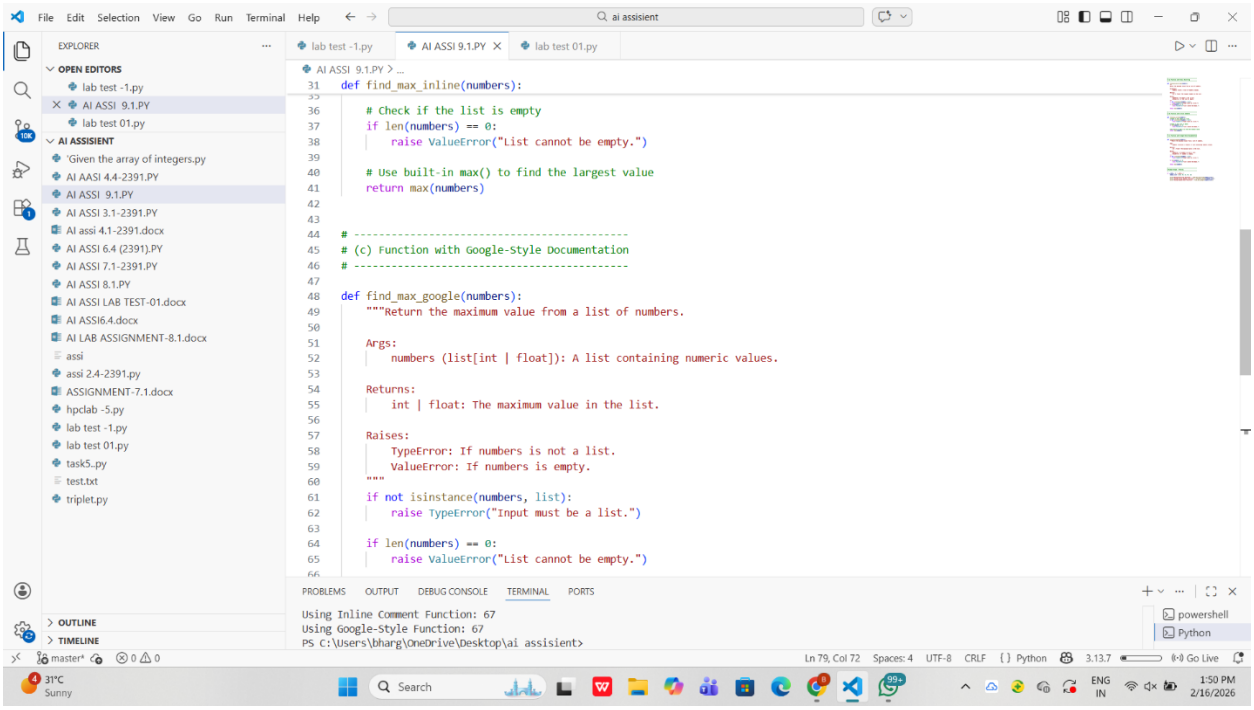
Problem 1:

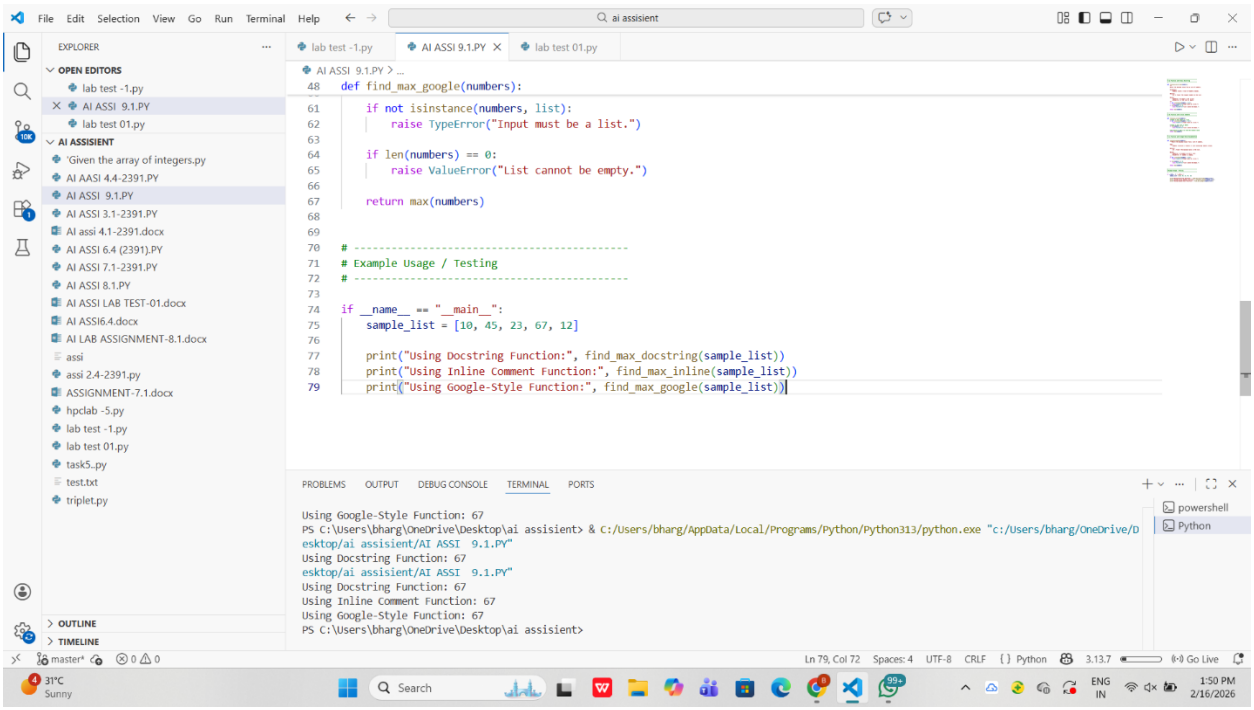
Consider the following Python function:

def find_max(numbers):

return max(numbers)







OBSERVATION:

1. The function was documented using docstring, inline comments, and Google-style format.

2. Docstrings allow automatic documentation generation using tools like pydoc.

3. Inline comments improve immediate code understanding but are not included in generated documentation.

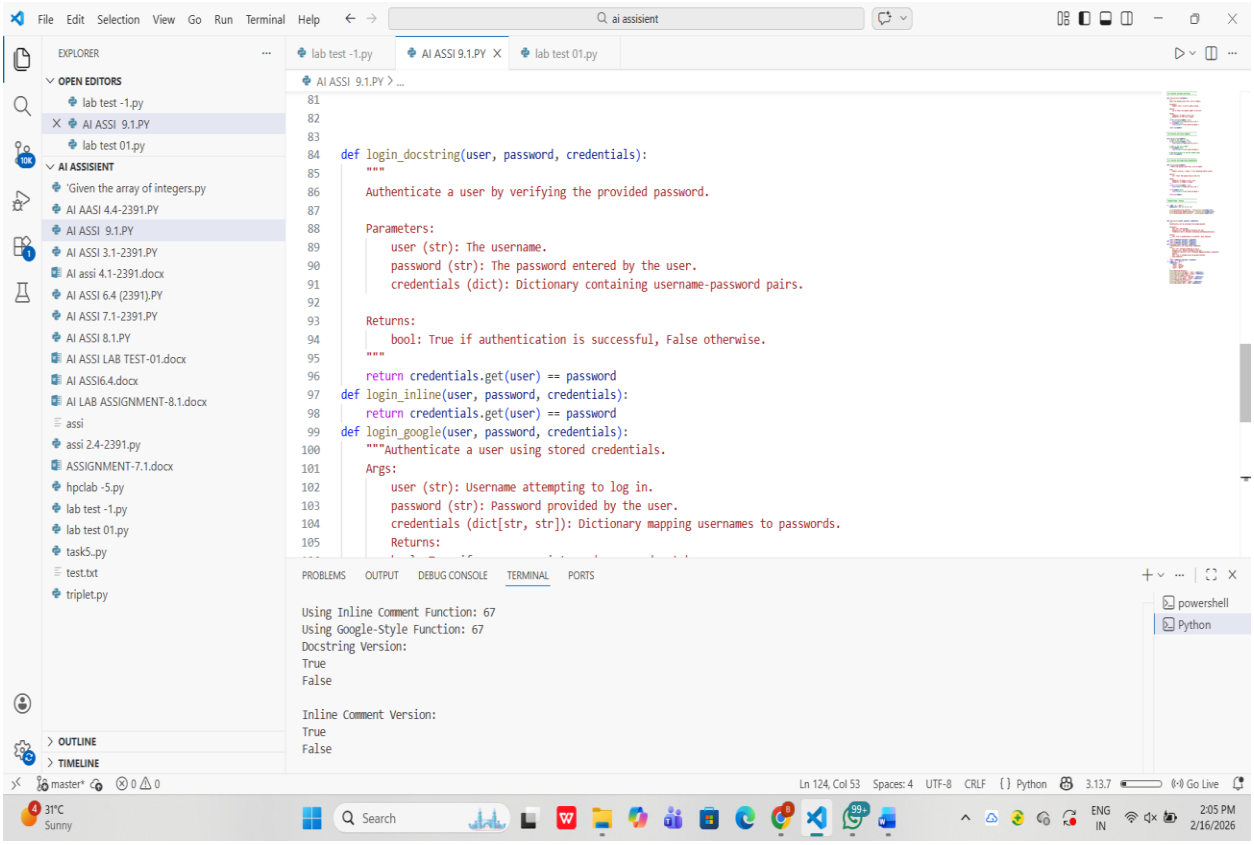
4. Google-style documentation provides clear structure with parameters and return details.

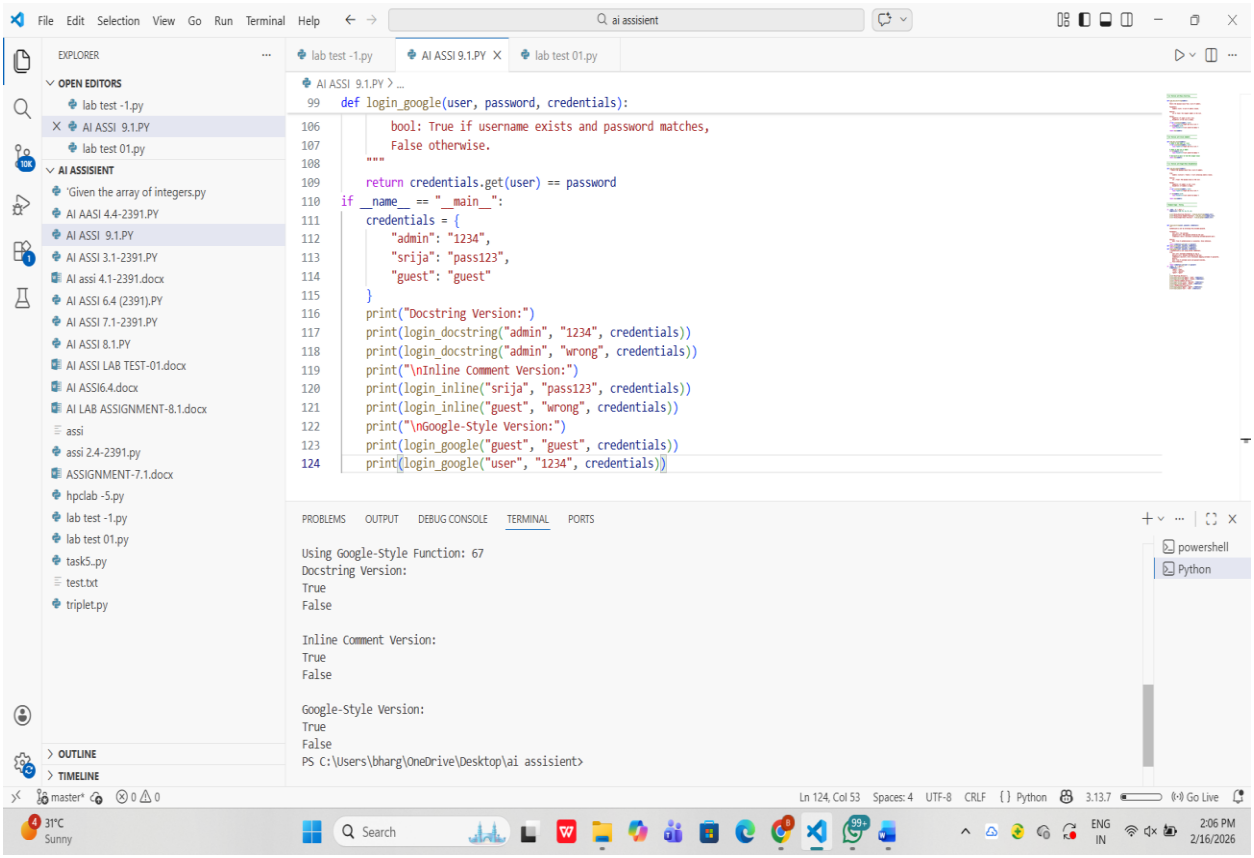
5. For a mathematical utilities library, Google-style docstrings are most effective due to clarity, consistency, and professional documentation standards.

Problem 2: Consider the following Python function:

def login(user, password, credentials):

return credentials.get(user) == password





OBSERVATION:

1. The login() function was documented in three formats: docstring, inline comments, and Google-style.

2. Docstrings support automatic documentation generation.

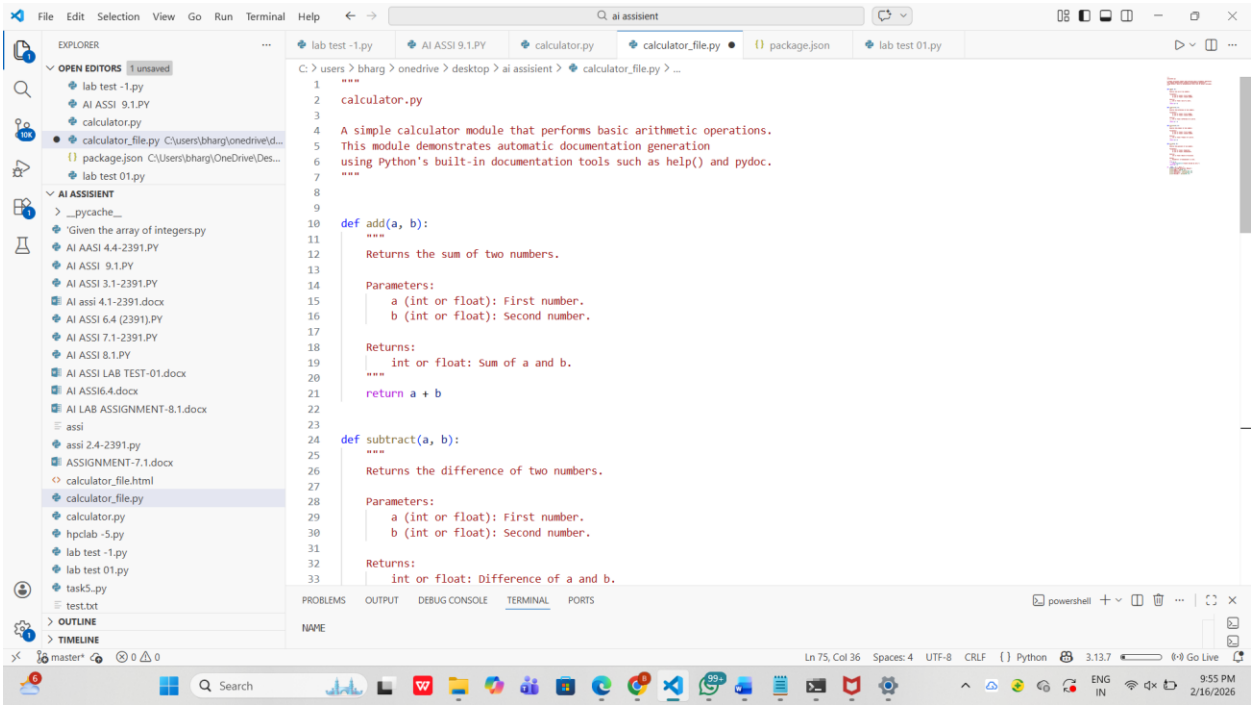
3. Inline comments improve readability but are not part of generated docs.

4. Google-style documentation is clear and well-structured.

5. Google-style is most helpful for new developers due to better clarity.

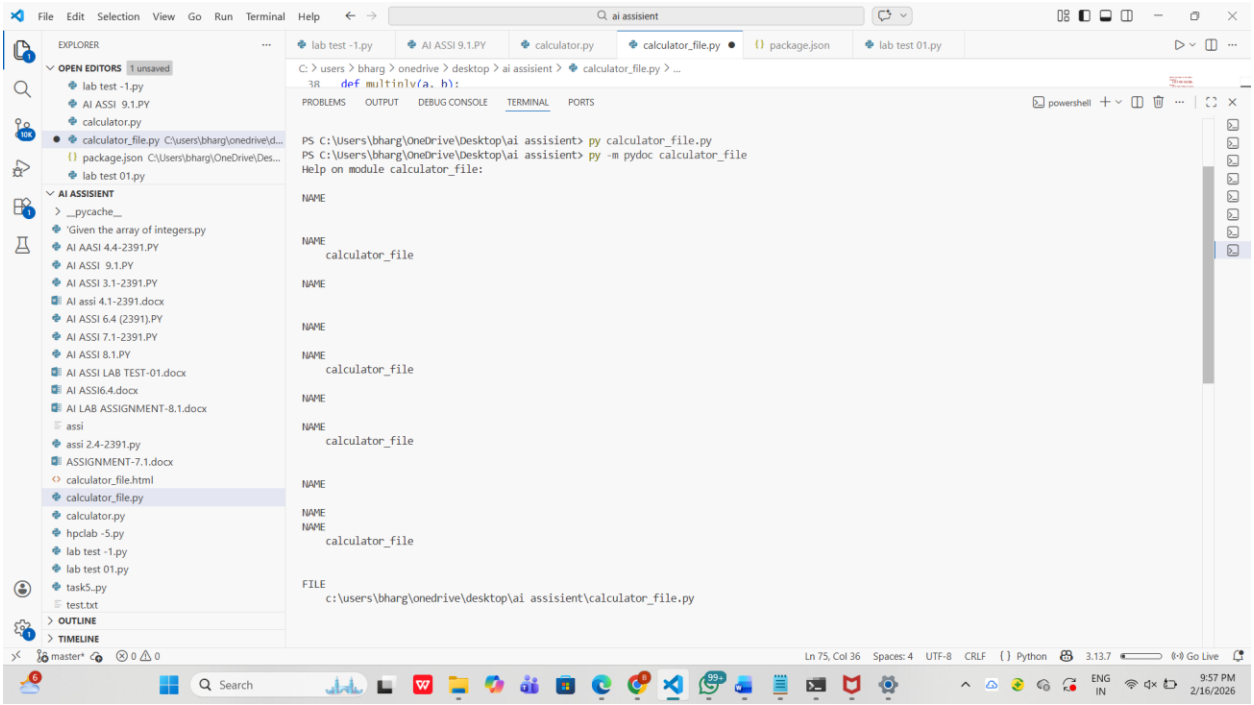
Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.



```python
def multiply(a, b):
        int or float: Product of a and b.
    """
    return a * b


def divide(a, b):
    """
    Returns the quotient of two numbers.

    Parameters:
        a (int or float): Numerator.
        b (int or float): Denominator.

    Returns:
        int or float: Result of division.

    Raises:
        ValueError: If denominator is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

if __name__ == "__main__":
    print("Testing Calculator Module")
    print("Add:", add(10, 5))
    print("Subtract:", subtract(10, 5))
    print("Multiply:", multiply(10, 5))
    print("Divide:", divide(10, 5))
```

```python
def add(a, b):
    """
    Returns the sum of two numbers.

    Parameters:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Sum of a and b.
    """
    return a + b


def subtract(a, b):
    """
    Returns the difference of two numbers.

    Parameters:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Difference of a and b.
    """
    return a - b


def multiply(a, b):
    """
    Returns the product of two numbers.

    Parameters:
        a (int or float): First number.
        b (int or float): Second number.

    Returns:
        int or float: Product of a and b.
    """
    return a * b


def divide(a, b):
    """
    Returns the quotient of two numbers.
```



OBSERVATION:

1. The calculator.py module was successfully created with the functions add(), subtract(), multiply(), and divide().

2. Proper docstrings were written for each function, enabling automatic documentation generation.

3. The module documentation was correctly displayed in the terminal using Python's built-in pydoc.

4. The HTML documentation file was successfully generated using the pydoc -w command.

5. The generated HTML file opened in the browser and clearly displayed module and function details.

Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

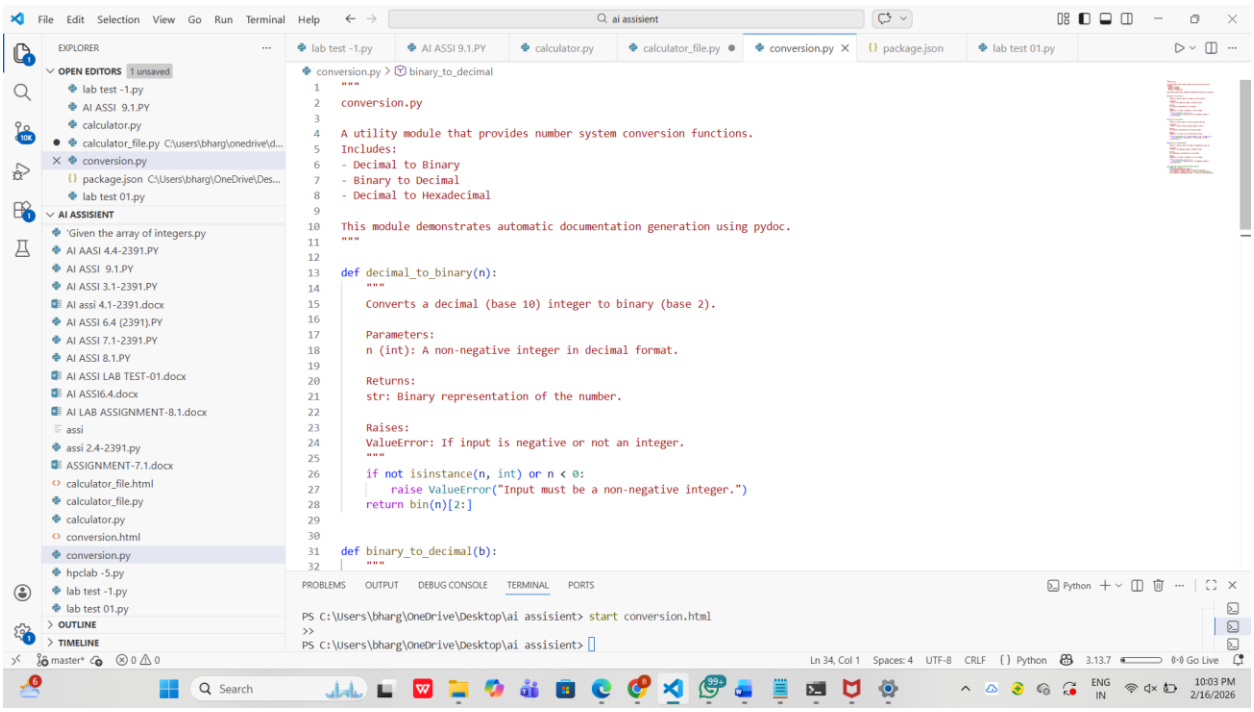
o decimal_to_hexadecimal(n)

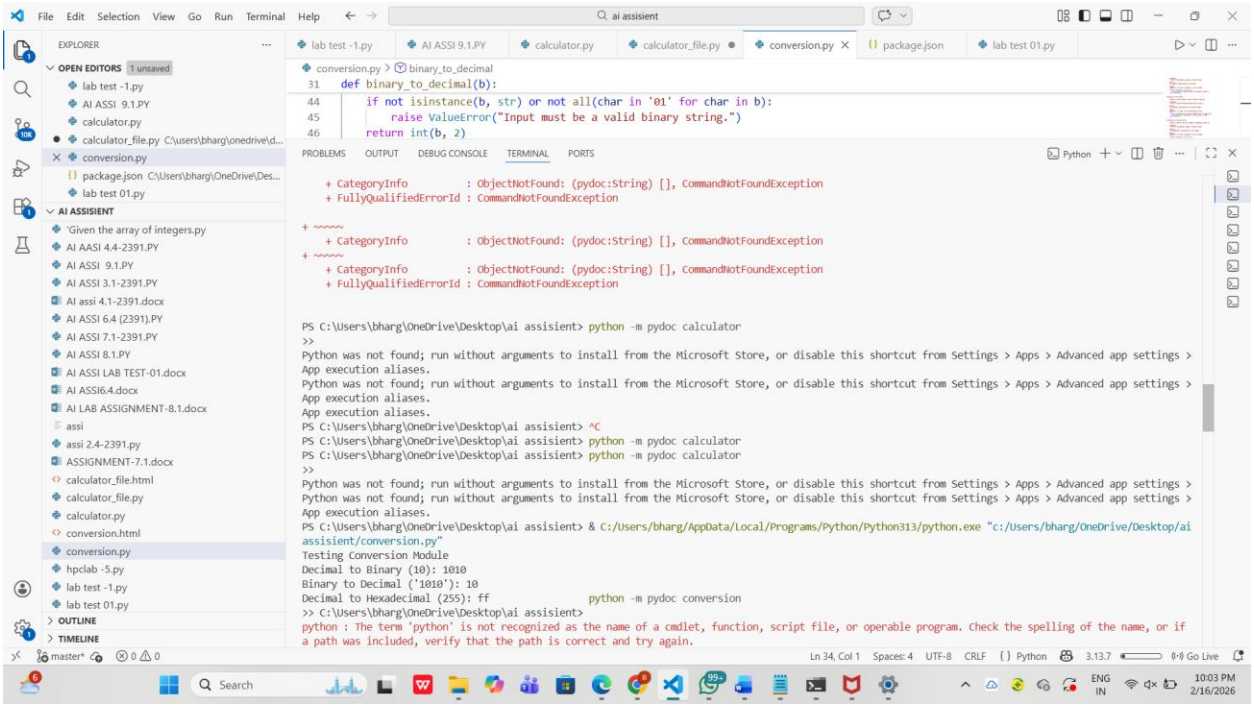
2. Use Copilot for auto-generating docstrings.

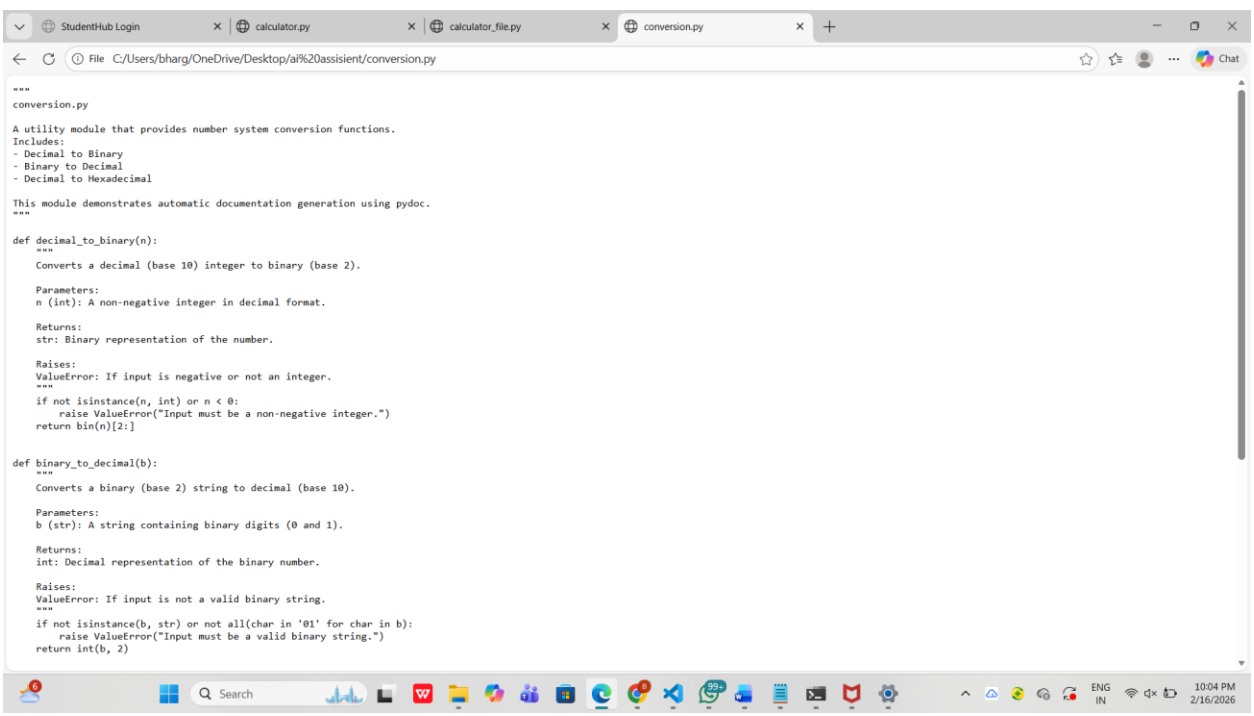
3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.




conversion.py > binary_to_decimal
31 def binary_to_decimal(b):
44 if not isinstance(b, str) or not all(char in '01' for char in b):
45 raise ValueError("Input must be a valid binary string.")
46 return int(b, 2)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
+ CategoryInfo : ObjectNotFound: (pydoc:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> python -m pydoc calculator
>>
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> ^C
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> python -m pydoc calculator
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> python -m pydoc calculator
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> & C:/Users/bharg/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/bharg/OneDrive/Desktop/ai assisient/conversion.py"
Testing Conversion Module
Decimal to Binary (10): 1010
Binary to Decimal ('1010'): 10
Decimal to Hexadecimal (255): ff
python : The term 'python' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.


File C:/Users/bharg/OneDrive/Desktop/ai%20assisient/conversion.py
"""
conversion.py
A utility module that provides number system conversion functions.
Includes:
- Decimal to Binary
- Binary to Decimal
- Decimal to Hexadecimal
This module demonstrates automatic documentation generation using pydoc.
"""
def decimal_to_binary(n):
"""
Converts a decimal (base 10) integer to binary (base 2).
Parameters:
n (int): A non-negative integer in decimal format.
Returns:
str: Binary representation of the number.
Raises:
ValueError: If input is negative or not an integer.
"""
if not isinstance(n, int) or n < 0:
raise ValueError("Input must be a non-negative integer.")
return bin(n)[2:]
def binary_to_decimal(b):
"""
Converts a binary (base 2) string to decimal (base 10).
Parameters:
b (str): A string containing binary digits (0 and 1).
Returns:
int: Decimal representation of the binary number.
Raises:
ValueError: If input is not a valid binary string.
"""
if not isinstance(b, str) or not all(char in '01' for char in b):
raise ValueError("Input must be a valid binary string.")
return int(b, 2)

OBSERVATION:

1 .The conversion.py module was successfully created with the required conversion functions.

2. GitHub Copilot assisted in generating clear and structured docstrings for each function.

3. All conversion functions worked correctly for valid inputs and handled invalid inputs safely.

4. Documentation was successfully displayed in the terminal using Python's built-in pydoc module.

5. The documentation was exported to an HTML file and opened in a web browser, confirming successful automatic documentation generation.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

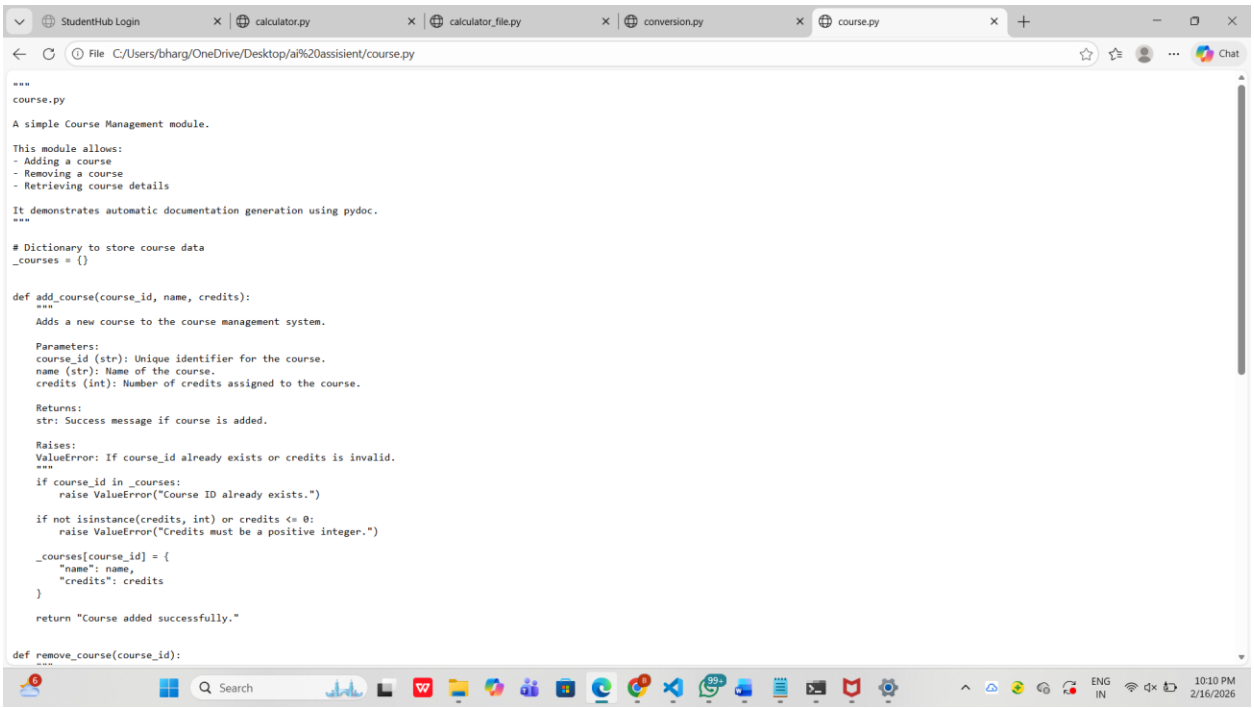
4. Export the documentation in HTML format and open it in a

browser.

```python
"""
course.py

A simple Course Management module.

This module allows:
- Adding a course
- Removing a course
- Retrieving course details

It demonstrates automatic documentation generation using pydoc.
"""

# Dictionary to store course data
_courses = {}


def add_course(course_id, name, credits):
    """
    Adds a new course to the course management system.

    Parameters:
    course_id (str): Unique identifier for the course.
    name (str): Name of the course.
    credits (int): Number of credits assigned to the course.

    Returns:
    str: Success message if course is added.
```

```
    + FullyQualifiedErrorId : CommandNotFoundException


Suggestion [3,General]: The command course.html was not found, but does exist in the current location. Windows PowerShell does not load commands from the
 current location by default. If you trust this cstart course.htmlype: ".\course.html". See "get-help about_Command_Precedence" for more details.
>> C:\Users\bharg\OneDrive\Desktop\ai assisient>
PS C:\Users\bharg\OneDrive\Desktop\ai assisient>
```

```
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> py course.py
Testing Course Module
Course added successfully.
{'name': 'Python Programming', 'credits': 4}
Course removed successfully.
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> & C:/Users/bharg/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/bharg/OneDrive/Desktop/ai
assisient/course.py"
Testing Course Module
Course added successfully.
{'name': 'Python Programming', 'credits': 4}
Course removed successfully.
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> py course.py
Testing Course Module
Course added successfully.
{'name': 'Python Programming', 'credits': 4}
Course removed successfully.
PS C:\Users\bharg\OneDrive\Desktop\ai assisient> py -m pydoc course
Help on module course:

NAME
    course - course.py

DESCRIPTION
    A simple Course Management module.

    This module allows:
    - Adding a course
    - Removing a course
    - Retrieving course details

    It demonstrates automatic documentation generation using pydoc.

FUNCTIONS
    add_course(course_id, name, credits)
        Adds a new course to the course management system.

        Parameters:
        course_id (str): Unique identifier for the course.
        name (str): Name of the course.
        credits (int): Number of credits assigned to the course.
```



```
"""
course.py

A simple Course Management module.

This module allows:
- Adding a course
- Removing a course
- Retrieving course details

It demonstrates automatic documentation generation using pydoc.
"""

# Dictionary to store course data
_courses = {}


def add_course(course_id, name, credits):
    """
    Adds a new course to the course management system.

    Parameters:
    course_id (str): Unique identifier for the course.
    name (str): Name of the course.
    credits (int): Number of credits assigned to the course.

    Returns:
    str: Success message if course is added.

    Raises:
    ValueError: If course_id already exists or credits is invalid.
    """
    if course_id in _courses:
        raise ValueError("Course ID already exists.")

    if not isinstance(credits, int) or credits <= 0:
        raise ValueError("Credits must be a positive integer.")

    _courses[course_id] = {
        "name": name,
        "credits": credits
    }

    return "Course added successfully."


def remove_course(course_id):
    """
```

OBSERVATION:

. The course.py module was successfully created with the required functions: add_course(), remove_course(), and get_course().

2.Proper docstrings were added to all functions, enabling automatic documentation generation.

3. The module handled errors effectively using appropriate exceptions (ValueError, KeyError).

4. Documentation was successfully generated in the terminal using the built-in Python module pydoc.

5. HTML documentation was generated and opened in a web browser, confirming that automatic documentation export works correctly.